

1 Physikalischer Kern: 2D-Strahlensegmentschnitt

Der rechnerische Kern der Physik-Engine basiert auf robuster linearer Algebra, um Randkollisionen zu lösen. Eine genaue mathematische Modellierung in diesem Stadium ist für hochauflösende akustische Simulationen entscheidend. Um den Rechendurchsatz zu maximieren, was bei der Berechnung von Millionen von Strahlen zwingend erforderlich ist, verzichtet die Engine auf rechnerisch aufwändige trigonometrische Funktionen zugunsten reiner Vektormathematik.

1.1 Geometrische Definitionen

Die beiden wichtigsten geometrischen Entitäten im 2D-Simulationsraum sind der akustische Strahl und die physikalische Grenze (die eine Wand darstellt).

Der Strahl Ein Schallstrahl wird durch einen Ursprungspunkt O und einen normalisierten Richtungsvektor definiert $\vec{D}$. Jeder Punkt P entlang der Strahlenbahn kann als Funktion der skalaren Distanz t entlang dieser Richtung berechnet werden:

$$P(t) = O + t\vec{D}$$

wobei $t \geq 0$ sicherstellt, dass der Strahl vom Emitter aus im Raum weitergeht.

Die Wand (Die Strecke) Eine Wand wird als Liniensegment dargestellt, das durch zwei Endpunkte, A und B , definiert ist. Jeder Punkt W auf diesem Segment wird durch Interpolation zwischen den Endpunkten mit einem skalaren Parameter u gefunden. Definiert man den Wandvektor als $\vec{V} = B - A$, ergibt sich die parametrische Gleichung durch:

$$W(u) = A + u\vec{V}$$

Damit ein Punkt strikt innerhalb der physikalischen Grenzen des Wandsegments liegt, muss die Bedingung $0 \leq u \leq 1$ erfüllt sein.

1.2 Schnittpunktberechnung

Um den Schnittpunkt zu bestimmen, werden die Strahlen- und Wandgleichungen gleichgesetzt:

$$O + t\vec{D} = A + u\vec{V}$$

Dies bildet ein System linearer Gleichungen, um t (Ausbreitungsdistanz) und u (Segmentschnittparameter) zu lösen. Durch die Definition eines Vektors $\vec{E} = A - O$, der vom Ursprung des Strahls zum Ausgangspunkt der Wand zeigt, ordnet sich die Gleichung um:

$$t\vec{D} - u\vec{V} = \vec{E}$$

Im zweidimensionalen Raum kann dieses System effizient mit dem 2D-Analogen des Kreuzprodukts gelöst werden (oft als Perp-Dot-Produkt bezeichnet). Für

zwei beliebige 2D-Vektoren $\vec{v}_1 = (x_1, y_1)$ und $\vec{v}_2 = (x_2, y_2)$ ergibt sich eine skalare Determinante: $\vec{v}_1 \times \vec{v}_2 = x_1 y_2 - y_1 x_2$.

Durch die Definition der Schnittdeterminante als $\det = \vec{D} \times \vec{V}$ können die Parameter t und u isoliert und algebraisch gelöst werden:

$$t = \frac{\vec{E} \times \vec{V}}{\det}$$

$$u = \frac{\vec{E} \times \vec{D}}{\det}$$

1.3 Schnittpunkt-Validierung (Trefferbedingungen)

Eine mathematisch gültige Lösung des linearen Systems garantiert nicht automatisch eine physikalische Kollision. Die folgenden Einschränkungen müssen überprüft werden, um eine gültige akustische Reflexion zu bestätigen:

- **Parallelität:** Wenn $\det \approx 0$, sind der Strahl und die Wand perfekt parallel und schneiden sich niemals. In der Implementierung ist ein kleiner Epsilon-Schwellenwert anstelle einer exakten Nullprüfung verwendet, um Gleitkommaraunauigkeiten zu vermeiden:

```
if det.abs() < 1e-6 {
    return None;
}
```

- **Richtungsabhängigkeit:** Wenn $t < 0$, schneiden sich die mathematischen Geraden, aber die Kollision erfolgt hinter dem Aussendungspunkt des Strahls:

```
if ray_t < 0.0 {
    return None;
}
```

- **Segmentgrenzen:** Wenn $u < 0$ oder $u > 1$, schneidet der Strahl zwar die unendliche Gerade, auf der die Wand liegt, verfehlt jedoch die physische Strecke

```
if wall_u < 0.0 || wall_u > 1.0 {
    return None;
}
```

1.4 Vektorreflexion

Nach der Bestätigung eines gültigen Schnittpunkts muss die resultierende Trajektorie der Schallwelle berechnet werden. Dies erfordert den Oberflächennormalenvektor

$\vec{N}$, der streng senkrecht von der Wand weg zeigt Für einen gegebenen 2D-Wandvektor $\vec{V} = (x, y)$ ist die normierte senkrechte Normale $\vec{N} = (-y, x)$.

Die Richtung des reflektierten Strahls $\vec{R}$ wird bestimmt, indem von der Richtung des einfallenden Strahls $\vec{D}$ das Doppelte der Projektion dieser Richtung auf die Oberflächennormale subtrahiert wird.

$$\vec{R} = \vec{D} - 2(\vec{D} \cdot \vec{N})\vec{N}$$

Innerhalb der Rust-Codebasis wird diese Berechnung der Vektorreflexion durch die `reflect` Methode optimiert, die vom `glam`-Crate bereitgestellt wird und Single Instruction, Multiple Data (SIMD)-Instruktionen nutzt, um diese Operationen mit minimalem Overhead durchzuführen.

1.5 Umstellung auf 3D: Vec3 und Möller-Trumbore-Algorithmus

Im Verlauf des Projekts wurde die ursprüngliche zweidimensionale Physik-Engine auf einen vollständig dreidimensionalen Simulationsraum erweitert. Ziel dieser Umstellung war es, Schallstrahlen nicht mehr ausschließlich in einer Ebene zu simulieren, sondern deren Ausbreitung in realistischen dreidimensionalen Räumen zu ermöglichen. Hierfür wurden die Datentypen der zentralen Simulationsobjekte (Ray, Mikrofonposition, Lautsprecherposition und Reflexionspunkte) von `Vec2` auf `Vec3` migriert. Während ein Punkt zuvor lediglich durch die Koordinaten (x, y) beschrieben wurde, enthält ein Punkt im dreidimensionalen Raum zusätzlich eine Höhenkoordinate z :

$$P = (x, y, z)$$

Die Erweiterung auf drei Dimensionen betrifft insbesondere die Positionen von Lautsprechern, Mikrofonen, Reflexionspunkten sowie die Richtungsvektoren der ausgesendeten Schallstrahlen.

1.5.1 Ablösung des 2D-Segment-Schnitts

Der bisher verwendete Strahl-Segment-Schnitt konnte ausschließlich Kollisionen mit zweidimensionalen Wandsegmenten berechnen. Für eine physikalisch korrekte Simulation in 3D müssen Schallstrahlen jedoch mit dreidimensionalen Flächen interagieren.

Aus diesem Grund werden Wände nicht mehr als Liniensegmente modelliert, sondern als Dreiecke. Dreiecke stellen die Standardrepräsentation polygonaler Oberflächen in der Computergrafik und Physiksimulation dar und ermöglichen die Darstellung beliebig komplexer Geometrien.

1.5.2 Möller-Trumbore-Algorithmus

Zur Berechnung von Strahl-Dreieck-Schnittpunkten wurde der Möller-Trumbore-Algorithmus implementiert. Dieser Algorithmus gehört zu den am häufigsten eingesetzten Verfahren im Raytracing, da er Schnittpunkte direkt berechnen kann, ohne zuvor die Ebenengleichung des Dreiecks bestimmen zu müssen. Gegeben sind:

- Ursprung des Strahls O
- Richtungsvektor des Strahls $\vec{D}$
- Die drei Eckpunkte des Dreiecks V_0 , V_1 und V_2

Zunächst werden die beiden Kantenvektoren des Dreiecks berechnet:

$$\vec{E}_1 = V_1 - V_0$$

$$\vec{E}_2 = V_2 - V_0$$

Anschließend wird mithilfe von Kreuz- und Skalarprodukten überprüft, ob der Strahl das Dreieck trifft und ob sich der Schnittpunkt innerhalb der Dreiecksgrenzen befindet.

Der Algorithmus liefert dabei drei wichtige Parameter:

- t : Entfernung des Schnittpunkts entlang des Strahls
- u : baryzentrische Koordinate innerhalb des Dreiecks
- v : baryzentrische Koordinate innerhalb des Dreiecks

Ein gültiger Schnittpunkt liegt nur dann vor, wenn folgende Bedingungen erfüllt sind:

$$t > 0$$

$$0 \leq u \leq 1$$

$$0 \leq v \leq 1$$

$$u + v \leq 1$$

Werden diese Bedingungen erfüllt, befindet sich der Schnittpunkt innerhalb des Dreiecks und eine Reflexion kann berechnet werden.

Der folgende Codeausschnitt zeigt den Kern des Möller-Trumbore-Schnittpunkttests. Die vollständige Implementierung befindet sich in `rust_service/src/geometry.rs` in der Funktion `check_intersection`.

```

let edge1 = triangle.v1 - triangle.v0;
let edge2 = triangle.v2 - triangle.v0;

let h = ray.direction.cross(edge2);
let a = edge1.dot(h);

if a.abs() < epsilon {
    return None;
}

let f = 1.0 / a;
let s = ray.origin - triangle.v0;
let u = f * s.dot(h);

if u < 0.0 || u > 1.0 {
    return None;
}

let q = s.cross(edge1);
let v = f * ray.direction.dot(q);

if v < 0.0 || u + v > 1.0 {
    return None;
}

let t = f * edge2.dot(q);

```

1.5.3 Reflexionsberechnung

Nach erfolgreicher Kollisionsprüfung wird die Oberflächennormale des Dreiecks bestimmt. Die Oberflächennormale wird zunächst normiert, um einen Einheitsnormalenvektor zu erhalten. Diese ergibt sich aus dem Kreuzprodukt der beiden Kantenvektoren:

$$\vec{N} = \vec{E}_1 \times \vec{E}_2$$

Die Richtung des reflektierten Strahls wird anschließend durch die allgemeine Reflexionsgleichung berechnet:

$$\vec{R} = \vec{D} - 2(\vec{D} \cdot \vec{N})\vec{N}$$

Dadurch entstehen physikalisch plausible Reflexionen an dreidimensionalen Oberflächen.

1.5.4 Ergebnis der Umstellung

Die erfolgreiche Migration von **Vec2** auf **Vec3** bildet die Grundlage für zukünftige Erweiterungen der Physik-Engine. Durch die Implementierung des Möller-Trumbore-Algorithmus können nun beliebige polygonale 3D-Oberflächen verarbeitet werden. Damit wurde die Grundlage geschaffen, realistische dreidimensionale Raumakustik mit beliebig komplexen Polygonnetzen zu simulieren. Die bisherige Beschränkung auf zweidimensionale Wandsegmente wurde vollständig aufgehoben.

2 Hochleistungs-Nebenläufigkeit und Datenparallelität

Ein wesentlicher Vorteil von Rust im High-Performance-Computing sind seine strengen Garantien zur Kompilierzeit gegen Data-Races. Die Verteilung einer rechenintensiven, iterativen Physikschleife auf Multi-Core-Prozessorarchitekturen erfordert in traditionellen Programmiersprachen oft komplexe Thread-Sperrlogiken (z.B. Mutexe), um Speicherbeschädigungen zu verhindern. In dieser Engine wird dieser Overhead vermieden durch ein funktionales, datenparalleles Paradigma unter Verwendung des **rayon**-Crates.

2.1 Architektonischer Wandel: Von Shared Mutation zu thread-lokalem-Folding

Bei einer standardmäßigen Single-Thread-Ausführung hängen sequenzielle Iterationen die berechneten Schnittpunktdateien (z.B. delay und pressure werte) an ein gemeinsames dynamisches Array an. Diese Schleife auf zahlreiche Threads aufzuteilen, würde zur gleichzeitigen Schreibversuchen auf dieselben Speicheradressen führen, was unmittelbare Data Races zur Folge hätte. Anstatt diese gemeinsam genutzt Arrays in rechenintensive, threadsichere Sperren (Locks) zu hüllen, nutzt die Architektur das Trait `into_par_iter()` von **rayon** in Kombination mit einer `fold`- und `reduce`- Pipeline.

Dieser Ansatz teilt die Arbeitslast sicher auf alle verfügbaren CPU-Kerne auf. Anstatt zu versuchen, in ein globales Array zu schreiben oder Speicher für jeden Strahl dynamisch zu allokalieren, verwendet die Engine die `fold`-Operation, um einen privaten, vorab allokalieren, thread-lokalen Puffer für jeden Kern bereitzustellen.

Jeder Thread simuliert unabhängig seine zugewiesene Teilmenge an Strahlen und hängt die Schnittpunktergebnisse sequenziell in seinen eigenen lokalen Speicherbereich an, ohne Sperren (lock-free). Sobald ein Thread seine Arbeitslast abgeschlossen hat, wird die `reduce`-Phase aktiviert, welche diese isolierten thread-lokalen Puffer sicher und effizient zu den endgültigen, vereinheitlichten Arrays zusammenführt. Dieser funktionale Ansatz garantiert die Datensicherheit und maximiert gleichzeitig die Hardwareauslastung.

```

use rayon::prelude::*;

fn run_simulation_parallel(
    config: &SimulationConfig,
    walls: &Vec<Wall>
) -> (Vec<f32>, Vec<f32>) {
    let (delays_par, pressures_par, _) = (0..config.rays_to_cast)
        .into_par_iter()
        // 1. Falten, Split
        .fold(|| {
            let local_delays = Vec::with_capacity(CAPACITY);
            let local_pressures = Vec::with_capacity(CAPACITY);
            let temp_ray_buffer = Vec::with_capacity(CAP);

            (local_delays, local_pressures, temp_ray_buffer)
        },
        |mut local_buf, _| {
            let fresh_ray = generate_initial_ray(config);
            simulate_single_ray(fresh_ray, walls, config, &mut local_buf.2);
            for (delay, pressure) in &local_buf.2 {
                local_buf.0.push(*delay);
                local_buf.1.push(*pressure);
            }
            local_buf
        })
        // 2. Reduzieren, Global Merge
    ).reduce(
        || (Vec::new(), Vec::new(), Vec::new()),
        |mut a, mut b| {
            a.0.append(&mut b.0);
            a.1.append(&mut b.1);
            a
        }
    );
    (delays_par, pressures_par)
}

```

3 Systemarchitektur und Pipeline-Orchestrierung

3.1 Python-Subprozess Runner und JSON-Ingestion

Neben der Rust-Physik-Engine wurde eine Python-Schnittstelle implementiert, um die vorkompilierte Rust-Binary automatisiert aus der Python-Pipeline heraus zu starten. Diese Schnittstelle befindet sich in der Datei `python.service/rust_runner.py` und bildet die Verbindung zwischen der performanten Rust-Simulation und den weiteren Verarbeitungsschritten in Python.

Die Funktion `run_rust_and_load_json` erhält den Pfad zur Rust-Binary sowie den erwarteten Pfad zur Ausgabedatei `ir_output.json`. Zunächst werden die übergebenen Pfade geprüft. Existiert die Rust-Binary nicht oder zeigt der Pfad nicht auf eine Datei, wird ein klarer Fehler ausgelöst. Dadurch werden fehlerhafte Konfigurationen früh erkannt.

Anschließend startet Python die Rust-Binary über das Modul `subprocess`. Die Rust-Simulation läuft dabei als externer Prozess. Python wartet, bis dieser Prozess vollständig beendet ist, und prüft danach den Rückgabecode. Falls die Rust-Binary mit einem Fehler beendet wird, werden sowohl `stdout` als auch `stderr` ausgegeben, um die Fehlersuche zu erleichtern.

Nach erfolgreicher Ausführung wird geprüft, ob die Datei `ir_output.json` tatsächlich erzeugt wurde. Diese Datei enthält die von Rust berechneten Impulsantwort-Daten, insbesondere die Verzögerungszeiten und Druckwerte der empfangenen Schallstrahlen. Die JSON-Datei wird anschließend geöffnet, mit `json.load` eingelesen und als Python-Dictionary zurückgegeben.

Der folgende Codeausschnitt zeigt den zentralen Ablauf der Schnittstelle:

```
result = subprocess.run(
    [str(rust_binary)],
    cwd=rust_service_dir,
    capture_output=True,
    text=True
)

if result.returncode != 0:
    raise RuntimeError("Rust-Binary ist fehlgeschlagen.")

with output_file.open("r", encoding="utf-8") as file:
    ir_data = json.load(file)

return ir_data
```

Die Kommunikation zwischen Rust und Python erfolgt über die Datei `ir_output.json`. Dadurch bleibt die Physik-Simulation in Rust, während Python die weitere Verarbeitung übernimmt. Die Lösung ist modular, einfach testbar und benötigt keine speziellen Bindings wie PyO3.

Die Funktion wurde anschließend mit der Testdatei `test_rust_runner_manual.py`

überprüft. Dabei wurde bestätigt, dass die von Rust erzeugte JSON-Datei erfolgreich geladen wird und die Datenstruktur die erwarteten Schlüssel `metadata` und `hits` enthält.

4 Leistungsoptimierungen und Profiling

Benchmarking the compiled binary (utilizing Rust's `--release` flag for maximum LLVM optimization) revealed significant bottlenecks and subsequent optimization opportunities within the parallelized architecture.

Das Benchmarking der kompilierten Binärdatei (unter Verwendung des `--release` flag und dem `let var = Instant::now();` timer) deckte signifikante Engpässe und darauffolgende Optimierungsmöglichkeiten innerhalb der parallelisierten Architektur auf.

4.1 Reduzierung des Speicherallokations-Overheads: `flat_map` vs. `fold`

Während die anfängliche parallele Implementierung den `flat_map`-iterator von `rayon` nutzt, offenbarte die Skalierung der Simulation auf 100.000.000 Strahlen einen signifikanten Leistungsengpass bezüglich der Speicherallokation. Die `flat_map`-Operation allozierte dynamisch einen neuen `Vec` für die Rückgabedaten jedes einzelnen Strahls. Bei der Verteilung auf alle CPU-Kerne führte dies zu Millionen von flüchtigen Heap-Allokationen pro Sekunde. Das System erlebte Memory Thrashing; die CPU-Auslastung fiel auf 0% der Hauptspeicher war ausgelastet, und das Betriebssystem entzog dem Ausführungs-Thread die Ressourcen, was zum Stillstand der Simulation führte.

Um diesen Allokations-Overhead zu beheben, wurde die parallele Pipeline refaktorisiert, um das `fold`-Muster zu verwenden. Anstatt Speicher pro Strahl zu allokalieren, stellt die `fold`-Operation exakt einen thread-lokalen Puffer (initialisiert über `Vec::with_capacity();`) pro CPU-Kern bereit. Die Kerne akkumulieren ihre lokalen Ergebnisse und führen diese erst nach Abschluss des Threads zusammen. Diese architektonische Änderung reduzierte die Heap-Allokationen um mehrere Größenordnungen und ermöglichte es dem System, 100.000.000 Strahlen effizient zu verarbeiten, ohne die Speicherressourcen zu erschöpfen.

4.2 Benchmarking Ergebnisse und I/O Overhead

Empirische Tests verdeutlichen die Wirksamkeit der optimierten Parallelisierung sowie die verborgenen Kosten von System-I/O und Thread-Initialisierung.

- **Paralleler Speedup:** Bei einer moderaten Arbeitslast wurde die Single-Thread-Simulation in 11,79 Sekunden abgeschlossen. Die durch `rayon` optimierte parallele Simulation bewältigt dieselbe Arbeitslast in 2,56 Sekunden, was einen $4,60\times$ Speedup auf der Test-Hardware ergab.

- **Thread-Pool-Kaltstart:** Bei extrem kleinen Arbeitslasten wies `rayon` einen Initialisierung-Overhead auf. In einem Benchmark mit wenigen Strahlen wurde die Single-Thread-Ausführung in 0,14 Sekunden abgeschlossen, während die parallele Ausführung 0,34 Sekunden in Anspruch nahm. Dies deutet darauf hin, dass die Kosten für die Initialisierung des Thread-Pools die Recheneinsparungen bei trivialen Arbeitslasten überwiegen
- **The I/O Engpass:** Das Profiling der End-to-End-Ausführungszeit deckte einen massiven Engpass bei der Datenserialisierung auf. In einem Test mit 10.000.000 Strahlen wurde die interne Simulation schnell ausgeführt (4,72s Single-Thread vs. 0,64s Parallel). Die anschließende Serialisierung und das Schreiben des Datensatzes dominierten jedoch die End-to-End-Laufzeit (37,40s vs. 33,24s), was die Notwendigkeit eines RAM-gestützten `BufWriter` unterstreicht.

5 Binaurale 3D-Audio-Ausgabe mit HRTF

Nach der Umstellung der Physik-Engine auf einen dreidimensionalen Raum wurde auch die Audioausgabe räumlich erweitert. Ziel ist es, aus einer trockenen Audiodatei ein Echo bzw. einen Nachhall zu erzeugen, dessen Reflexionen über Kopfhörer räumlich wahrgenommen werden können. Dafür reicht eine klassische Impulsantwort aus Delay- und Amplitudenwerten nicht mehr aus. Zusätzlich muss bekannt sein, aus welcher Richtung eine Reflexion am Mikrofon eintrifft.

Die Rust-Simulation speichert deshalb pro Mikrofontreffer drei Werte: `delay`, `pressure` und `direction`. `delay` beschreibt die Ankunftszeit, `pressure` die verbleibende Energie und `direction` die räumliche Einfallsrichtung. Damit wird aus der bisherigen eindimensionalen Impulsantwort eine gerichtete Impulsantwort.

5.1 Richtungsinformationen aus der Simulation

Die Richtungsinformationen werden direkt beim Mikrofontreffer berechnet. Sobald ein Ray-Segment nahe genug am Mikrofon vorbeifliegt, wird ein normalisierter Vektor vom Mikrofon zum Trefferpunkt gespeichert. Dieser Richtungsvektor enthält keine Distanz mehr, sondern nur die Information, aus welcher Richtung der Schallanteil kommt.

```
let direction = (end_point - config.mic_position).normalize();

ray_hits.push((
    total_distance_at_hit / 343.0,
    current_pressure,
    direction
));
```

Da die Simulation parallel arbeitet, wird diese zusätzliche Information zusammen mit den Delay- und Pressure-Werten gesammelt. Auch der JSON-Export wurde entsprechend erweitert, sodass `ir_output.json` neben `delays_seconds` und `pressures` auch eine Liste `directions` enthält. Diese gerichtete Impulsantwort bildet die Grundlage für die HRTF-Verarbeitung in Python.

5.2 HRTF als Grundlage räumlicher Wahrnehmung

Für die binaurale Wiedergabe wird eine HRTF-Datenbank verwendet. HRTF steht für *Head-Related Transfer Function* und beschreibt, wie ein eintreffender Schall durch Kopf, Ohrmuscheln und Oberkörper abhängig von seiner Richtung verändert wird. Diese Filtereffekte sind entscheidend dafür, dass Menschen Schallquellen räumlich lokalisieren können.

In der Implementierung wird eine SOFA-Datei aus der CIPIC-Datenbank verwendet, beispielsweise `subject_003.sofa`. SOFA ist ein standardisiertes Format für räumliche Audiodaten. Die Klasse `HRTFDatabase` lädt daraus die Messpositionen und die dazugehörigen HRIRs. HRIR steht für *Head-Related*

Impulse Response und bezeichnet die Impulsantworten für linkes und rechtes Ohr zu einer bestimmten Einfallsrichtung.

Für jede simulierte Reflexion wird aus dem Richtungsvektor zunächst ein Winkelpaar aus Azimut und Elevation berechnet. Der Azimut beschreibt die horizontale Richtung, also vorne, links, rechts oder hinten. Die Elevation beschreibt die vertikale Richtung, also oben oder unten. Anhand dieser Winkel wird anschließend der nächstgelegene Messpunkt in der HRTF-Datenbank ausgewählt.

```
az, el = vector_to_az_el(direction)
hrir_l, hrir_r = hrtf_db.get_closest_hrir(az, el)
```

Diese Version verwendet dafür eine einfache Nearest-Neighbor-Auswahl. Das bedeutet, dass nicht zwischen Messpunkten interpoliert wird, sondern immer die nächstgelegene vorhandene HRIR genutzt wird. Dadurch bleibt die Implementierung robust und leicht nachvollziehbar.

5.3 Binaurale Faltung

Die HRTF-basierte Faltung erfolgt in der Klasse `BinauralConvolver`. Für jeden Ray-Hit wird anhand seiner Einfallsrichtung eine passende HRIR für das linke und rechte Ohr gewählt. Der Delay-Wert bestimmt, an welcher Stelle diese HRIR in die binaurale Impulsantwort eingesetzt wird. Der Pressure-Wert skaliert die Stärke der Reflexion.

Dadurch wird jede Reflexion nicht nur zeitlich verzögert und abgeschwächt, sondern zusätzlich durch einen richtungsabhängigen Ohrfilter geprägt. Aus allen Ray-Hits entsteht eine zweikanalige binaurale Impulsantwort. Diese wird anschließend mit dem trockenen Eingangssignal gefaltet:

```
out_l = signal.oaconvolve(dry_audio, binaural_ir[:, 0], mode="full")
out_r = signal.oaconvolve(dry_audio, binaural_ir[:, 1], mode="full")
wet = np.stack([out_l, out_r], axis=1)
```

Das Ergebnis ist eine Stereo-WAV-Datei. Über Kopfhörer kann diese Datei als räumliches Echo bzw. räumlicher Nachhall wahrgenommen werden.

5.4 Integration und Ergebnis

Die Integration erfolgt in der bestehenden Python-Pipeline. Nach dem Start der Rust-Simulation wird die erzeugte Datei `ir_output.json` geladen. Enthält sie Richtungsdaten, wird die HRTF-basierte binaurale Faltung verwendet. Falls keine Richtungsdaten vorhanden sind, bleibt die ursprüngliche Faltung als Fallback erhalten.

Der entscheidende Unterschied zu einfachem Stereo-Panning besteht darin, dass HRTF nicht nur Lautstärkeunterschiede zwischen linkem und rechtem Kanal erzeugt. Stattdessen werden frequenzabhängige Filtereffekte berücksichtigt, die beim realen Hören durch Kopf und Ohrmuscheln entstehen. Dadurch können Reflexionen über Kopfhörer räumlich differenzierter wahrgenommen werden.

Die Erweiterung macht aus der klassischen Impulsantwort eine gerichtete binaurale Impulsantwort. Damit wird das simulierte Echo nicht nur mit korrekter Verzögerung und Lautstärke, sondern auch räumlich wiedergegeben.